BrowserSession and the browser-state summary

Week 3, Lecture 1 · Browser Use: How LLM Agents Drive the Web

Summer 2026 · Capturing browser state: the session layer

What we'll cover

- What a BrowserSession owns and manages
- The four fields of the browser-state summary
- Playwright for launch and action vs CDP for DOM intelligence
- How the DOM service produces the element tree and selector map
- Key BrowserSession configuration options
- How the state summary feeds the agent loop

What a BrowserSession is

The session owns the browser

A `BrowserSession` is the object that owns and controls the browser process.

- Holds the Playwright browser and context (cookies, storage, permissions)
- Holds the active page handle
- Is the gateway for all perception: DOM extraction, screenshots
- Is the gateway for all action: clicks, typing, navigation

When you give an agent a task, it uses its session to see the page and to act on it. The LLM never touches the browser directly.

(browser-use team, 2026)

Configuring a session

Common options at construction time:

```
from browser_use import BrowserSession

# Development: visible browser, kept alive
session = BrowserSession(
    headless=False,
    keep_alive=True,
)

# Production: headless, saved profile
session = BrowserSession(
    headless=True,
    user_data_dir="./profiles/mysite",
)
```

`headless=False` is for debugging. `user_data_dir` lets the agent reuse cookies and login sessions. `keep_alive=True` keeps the browser open between agent runs.

The browser-state summary

Four fields, three always present

Each agent step begins with a state capture. The result is a browser-state summary with four fields:

1. **url**: the current page URL
2. **title**: the page title from the browser tab
3. **Serialized DOM**: the filtered, indexed element tree plus the selector map
4. **Screenshot**: a viewport PNG (optional, only when `use_vision=True`)

The LLM receives fields 1, 2, and 3 as text. Field 4 is passed as a vision input when present. The selector map is used internally by the framework for action execution.

What the element tree looks like

The DOM service produces a compact text representation of the page:

```
[0] Link: "Hacker News" (nav)
[1] Link: "new" (nav)
[2] Link: "past" (nav)
[3] Link: "Story title: An interesting article" (main)
[4] Button: "vote" (main)
[5] Link: "82 comments" (main)
...
```

Each numbered item is one interactive element the agent can act on by index. Static paragraphs, hidden elements, and off-viewport content are excluded.

(browser-use contributors, 2026)

Playwright and CDP: two distinct roles

Division of labor

Role	Technology
Launch and lifecycle	Playwright
Navigate, click, type, scroll	Playwright
DOM snapshot extraction	CDP (Chrome DevTools Protocol)
Accessibility tree	CDP
Visibility and interactivity detection	CDP

Playwright provides a high-level API that is reliable and cross-browser. CDP gives the DOM service direct access to the browser's internal snapshot: one round trip, full structure, computed layout.

Why not use Playwright's DOM inspection?

Playwright exposes `page.accessibility.snapshot()` and similar methods. The DOM service uses CDP directly because:

- CDP's `DOMSnapshot.captureSnapshot` returns the full rendered DOM, computed positions, and ARIA roles in one call
- Visibility filtering (is this element in the viewport?) requires layout coordinates that CDP provides and Playwright's high-level API does not expose directly
- One CDP call produces everything needed; Playwright would require multiple calls

The result: a faster, more complete snapshot that includes exactly what the filtering logic needs.

(browser-use contributors, 2026)

The DOM service: from raw DOM to numbered elements

Three filters, one index

The DOM service applies three filters before numbering elements:

1. **Interactivity:** keep links, buttons, inputs, selects, click-handler elements, and ARIA interactive roles. Drop static text, decorative images, layout containers.
2. **Visibility:** drop elements hidden by CSS, off-screen, or clipped to zero size. The agent can only act on what a user could see.
3. **Sequential index:** assign integers 0, 1, 2, … to every element that passed the first two filters.

A page with 400 DOM nodes typically yields 30-80 indexed elements.

The selector map

Every integer index maps to a real CSS locator stored in the `selector_map`:

```
{
  0: "a[href='/']",
  1: "a[href='/news']",
  3: "a[href='https://example.com/story-title']",
  4: ".votelinks button",
  5: "a[href='item?id=12345']",
  ...
}
```

The LLM says "click element 5." The framework looks up index 5 in the selector map and passes that locator to Playwright to execute the click.

The model never writes CSS selectors. It uses integers.

> The DOM gives you a structured, semantic map of the page. A screenshot gives you a picture. For navigation and interaction, the map wins.

browser-use contributors (2026)

Take-aways

1. A `BrowserSession` owns the browser process, context, and active page. All perception and action flows through it.
2. Playwright handles browser launch and action execution; CDP handles DOM extraction. The split gives each layer the access it needs.
3. The browser-state summary has four fields: URL, title, serialized DOM with selector map, and an optional screenshot.
4. The DOM service filters to interactive, visible elements and assigns sequential integer indices. The agent's vocabulary is these integers.
5. The selector map translates model-chosen indices back to real CSS locators at execution time.

What's next

- **Lecture 2 (this week):** Fitting the page into a prompt: context budget, ephemeral messages, token cost of vision vs DOM
- **Reading:** Week 3 reading at `/c/browser-use-26su/readings/wk03` (required before section)
- **Section this week:** Configure two sessions, capture state summaries, compare outputs
- **HW2:** Trace the state-to-action pipeline (assigned this week)